

A Minimal Copper-Plate Storage-Scheduling Benchmark for Differential Dynamic Programming

Aryan Ritwajeet Jha, *Student Member, IEEE*

I. COPPER-PLATE STORAGE-SCHEDULING MODEL

A. Optimization Problem

A single storage device and a single grid connection serve a known demand p_L^t over T periods of length Δt , all quantities in per unit. Following the sign convention of [1], $P_B^t > 0$ discharges and $P_B^t < 0$ charges; B^t is the energy at the *start* of period t , so B^1 is given and B^{T+1} is terminal. With $t \in \mathcal{T} = \{1, \dots, T\}$:

$$\min_{P_{\text{Subs}}, P_B, B} \sum_{t=1}^T \left[c^t P_{\text{Subs}}^t + C_B (P_B^t)^2 \right] \Delta t + w (B^{T+1} - B^1)^2 \quad (1a)$$

$$\text{s.t. } P_{\text{Subs}}^t + P_B^t - p_L^t = 0, \quad t \in \mathcal{T} \quad (1b)$$

$$B^1 = B_0 \quad (1c)$$

$$B^{t+1} = B^t - P_B^t \Delta t, \quad t \in \mathcal{T} \quad (1d)$$

$$P_{\text{Subs}}^t \geq 0, \quad t \in \mathcal{T} \quad (1e)$$

$$P_B^{\min} \leq P_B^t \leq P_B^{\max}, \quad t \in \mathcal{T} \quad (1f)$$

$$B^{\min} \leq B^{t+1} \leq B^{\max}, \quad t \in \mathcal{T} \quad (1g)$$

The *base instance* is (1a)–(1d), with all three bounds dropped.

B. Optimal-Control Form

Restated over $N = T$ stages in the form required by DDP, with a distinct cost and constraint at each stage:

$$\begin{aligned} \min_{x, u} \quad & \sum_{t=1}^{N-1} \ell^t(x^t, u^t) + \ell^F(x^N, u^N) \\ \text{s.t. } \quad & x^1 = \hat{x}^1, \\ & x^{t+1} = f(x^t, u^t), \quad t \in \{1, \dots, N-1\}, \\ & c^t(x^t, u^t) = 0, \quad t \in \{1, \dots, N\}. \end{aligned} \quad (2)$$

The state is the stored energy alone; the controls are the two power set-points:

$$x^t = B^t \in \mathbb{R}, \quad u^t = (P_{\text{Subs}}^t, P_B^t)^\top \in \mathbb{R}^2, \quad (3)$$

with $\hat{x}^1 = B_0$. Because stage t carries its own c^t and p_L^t , the per-period data enters directly and nothing is interpolated:

$$f(x, u) = x - u_2 \Delta t, \quad (4)$$

$$c^t(x, u) = u_1 + u_2 - p_L^t, \quad (5)$$

$$\ell^t(x, u) = [c^t u_1 + C_B u_2^2] \Delta t, \quad (6)$$

$$\ell^F(x, u) = [c^T u_1 + C_B u_2^2] \Delta t + w (x - u_2 \Delta t - B_0)^2. \quad (7)$$

Bounds (1e)–(1f) map onto control limits $b^L \leq u^t \leq b^U$. The state bound (1g) is appended as a second row of c^t via a slack s^t adjoined to the control, giving $n_c = 2$, $n_u = 3$:

$$(x - u_2 \Delta t) - B^{\min} - s^t = 0, \quad 0 \leq s^t \leq B^{\max} - B^{\min}. \quad (8)$$

Remark. A per-stage ℓ^t and c^t is what the method assumes — the objective, dynamics and constraint functions are time-varying in general — but the reference implementation exposed only one of each for the whole horizon. An earlier revision of this benchmark worked around that by adjoining a time index τ to the state and recovering c^t and p_L^t from a degree- $(T-1)$ Lagrange interpolant of it. That device is exact at the nodes and was numerically well behaved past $T = 24$, but its symbolic expression grows with the horizon and exhausted the code generator at $T = 48$. Restoring per-stage data removes it, and with it the extra state: n_x falls from 2 to 1 and the horizon ceiling disappears.

C. What the Solver Actually Forms

Section I-B states the model in the solver's generic objects x, u, ℓ, f, c . This subsection eliminates them: every quantity below is written in the variables of (1) and the multipliers attached to its constraints. All expressions are verified against the derivative functions the solver actually builds.

1) Variables carried at stage t

The state is the stored energy, and nothing else:

$$\underbrace{B^t}_{\text{state}}; \quad \underbrace{P_{\text{Subs}}^t, P_B^t, s^t}_{\text{controls}}. \quad (9)$$

Each stage carries one multiplier per constraint:

$$\varphi_{\text{bal}}^t : P_{\text{Subs}}^t + P_B^t - p_L^t = 0 \quad (10)$$

$$\varphi_{\text{soc}}^t : (B^t - P_B^t \Delta t) - B^{\min} - s^t = 0 \quad (11)$$

$$\nu_{\text{Subs}}^t : P_{\text{Subs}}^t \geq 0 \quad (12)$$

$$\nu_{\text{min}}^t, \nu_{\text{max}}^t : P_B^{\min} \leq P_B^t \leq P_B^{\max} \quad (13)$$

$$\lambda_{B \text{ min}}^t, \lambda_{B \text{ max}}^t : 0 \leq s^t \leq B^{\max} - B^{\min}. \quad (14)$$

The last line is the correspondence that matters: because s^t is defined by (11) as the slack of B^{t+1} above $B^{\min}$, its two box multipliers *are* the energy-bound multipliers, $\lambda_{B \text{ min}}^t$ on $B^{t+1} \geq B^{\min}$ and $\lambda_{B \text{ max}}^t$ on $B^{t+1} \leq B^{\max}$. There is no separate state-constraint dual. P_{Subs}^t has no upper bound, so no multiplier is attached to one.

2) Stage derivatives

With the dynamics $B^{t+1} = B^t - P_B^t \Delta t$ linear and the constraints (10)–(11) linear in the controls, every second derivative of f and of c^t vanishes, as does the cross term

$\partial^2 \ell^t / \partial u \partial B$. What remains is

$$\frac{\partial c}{\partial (P_{\text{Subs}}, P_B, s)} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & -\Delta t & -1 \end{bmatrix} \quad (15)$$

$$\frac{\partial c}{\partial B} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad (16)$$

$$\frac{\partial B^{t+1}}{\partial P_B^t} = -\Delta t \quad (17)$$

$$\frac{\partial^2 \ell}{\partial (P_{\text{Subs}}, P_B, s)^2} = \text{diag}(0, 2C_B \Delta t, 0). \quad (18)$$

The terminal stage adds $2w$ to the B curvature, $2w(\Delta t)^2$ to the P_B curvature, and $-2w\Delta t$ to their cross term.

3) The matrix that is factorised

Write V_{BB}^{t+1} for the curvature of the cost-to-go in the stored energy. Collecting (17)–(18) with the barrier terms, the stage Hessian is *diagonal*:

$$\hat{H}^t = \text{diag} \left(\underbrace{\frac{\nu_{\text{Subs}}^t}{P_{\text{Subs}}^t}}_{P_{\text{Subs}}}, \underbrace{2C_B \Delta t + (\Delta t)^2 V_{BB}^{t+1} + \Sigma_{P_B}^t}_{P_B}, \underbrace{\Sigma_s^t}_s \right), \quad (19)$$

$$\Sigma_{P_B}^t = \frac{\nu_{\min}^t}{P_B^t - P_B^{\min}} + \frac{\nu_{\max}^t}{P_B^{\max} - P_B^t}, \quad (20)$$

$$\Sigma_s^t = \frac{\lambda_{B\min}^t}{s^t} + \frac{\lambda_{B\max}^t}{B^{\max} - B^{\min} - s^t}. \quad (21)$$

Three controls and two equalities leave a *one*-dimensional null space, so after eliminating (15) exactly one direction survives:

$$d = (\Delta P_{\text{Subs}}, \Delta P_B, \Delta s) = (-1, +1, -\Delta t), \quad (22)$$

which is the statement that the only freedom left at a stage is to move power between the substation and the battery, the energy slack following. The quantity the solver factorises is therefore the *scalar*

$$d^\top \hat{H}^t d = \frac{\nu_{\text{Subs}}^t}{P_{\text{Subs}}^t} + 2C_B \Delta t + (\Delta t)^2 V_{BB}^{t+1} + \Sigma_{P_B}^t + (\Delta t)^2 \Sigma_s^t, \quad (23)$$

and the Cholesky whose failure triggers regularisation is a positivity test on this one number. Dropping the energy bound removes s^t and its two multipliers, leaving $d = (-1, +1)$ and $d^\top \hat{H}^t d = \nu_{\text{Subs}}^t / P_{\text{Subs}}^t + 2C_B \Delta t + (\Delta t)^2 V_{BB}^{t+1} + \Sigma_{P_B}^t$; the base instance strips the barrier terms too, leaving $2C_B \Delta t + (\Delta t)^2 V_{BB}^{t+1}$, which is where C_B enters as the curvature that keeps the problem solvable.

4) What is passed backward

Let V_B^t denote the gradient of the cost-to-go in the stored energy. With a one-dimensional state, V_B^t and V_{BB}^t are scalars, so the whole backward recursion is scalar arithmetic. The backward recursion carried between stages is

$$V_B^t = V_B^{t+1} + \varphi_{\text{soc}}^t + (\beta^\top \hat{Q}_u^t)_B + (\omega^\top c^t)_B, \quad (24)$$

$$V_{BB}^t = V_{BB}^{t+1} + (\beta^\top \hat{B}^t)_{BB} + (\omega^\top c_x^t)_{BB}, \quad (25)$$

where β and ω are the feedback gains produced by the stage solve. At a feasible iterate ($c^t = 0$) that has converged ($\hat{Q}_u^t = 0$), (24) collapses to

$$V_B^t - V_B^{t+1} = \varphi_{\text{soc}}^t, \quad (26)$$

i.e. the value gradient in stored energy is the discrete costate, and its stage-to-stage increment is exactly the multiplier on the energy relation. Equation (25) has no counterpart in a first-order method: it is the curvature, and it is the object that a scheme passing only (26) between stages does not carry.

5) What is passed forward

With step size γ and $\delta B = B^t - \bar{B}^t$ the deviation from the nominal trajectory, every quantity is updated by the same affine rule:

$$P_{\text{Subs}}^t = \bar{P}_{\text{Subs}}^t + \gamma \alpha_{\text{Subs}}^t + \beta_{\text{Subs}}^t \delta B, \quad P_B^t = \bar{P}_B^t + \gamma \alpha_B^t + \beta_B^t \delta B, \quad (27)$$

$$\varphi_\bullet^t = \bar{\varphi}_\bullet^t + \gamma \psi_\bullet^t + \omega_\bullet^t \delta B, \quad \nu_\bullet^t = \bar{\nu}_\bullet^t + \gamma \chi_\bullet^t + \zeta_\bullet^t \delta B, \quad (28)$$

so the primal variables, the equality multipliers and the bound multipliers are all propagated together along the same search direction, with a single γ accepted by the filter for all of them.

D. Closed-Form Reference

The base instance is a strictly convex equality-constrained QP. Eliminating $P_{\text{Subs}}^t = p_L^t - P_B^t$ and $B^{T+1} - B^1 = -\Delta t \sum_t P_B^t$ reduces it to

$$J(P_B) = \text{const} - \Delta t \sum_t c^t P_B^t + C_B \Delta t \sum_t (P_B^t)^2 + w(\Delta t)^2 s^2, \quad (29)$$

with $s = \sum_t P_B^t$. Stationarity divided through by Δt is $-c^k + 2C_B P_B^k + 2w\Delta t s = 0$; summing over k collapses it to

$$s = \frac{\sum_t c^t}{2C_B + 2Tw\Delta t}, \quad P_B^k = \frac{c^k - 2w\Delta t s}{2C_B}. \quad (30)$$

E. A First-Order Differential Dynamic Programming Scheme

The same problem can be attacked by a first-order variant of differential dynamic programming: rather than a Riccati sweep carrying both derivatives of the cost-to-go, it carries only the first, and solves one small program per period sweeping forward. Write k for the sweep index. Here $B[t]$ is the energy *after* period t , with $B[0] = B_0$, so $B[T]$ is the terminal energy B^{T+1} of (1).

1) The subproblem at stage t

Sweep k solves, for $t = 1, \dots, T$ in order,

$$\min_{P_{\text{Subs}}[t], P_B[t], B[t], P_B[t+1]} c^t P_{\text{Subs}}[t] \Delta t + C_B (P_B[t])^2 \Delta t + \mu^{k-1}[t+1] (B^{k-1}[t+1] - B[t] + \Delta t P_B[t+1]) \quad (31)$$

subject to the period's own balance, dynamics and limits,

$$P_{\text{Subs}}[t] + P_B[t] = p_L^t \quad (32)$$

$$B[t] - B^k[t-1] + \Delta t P_B[t] = 0 \quad (33)$$

$$B^{\min} \leq B[t] \leq B^{\max}, \quad P_B^{\min} \leq P_B[t] \leq P_B^{\max}. \quad (34)$$

Two couplings appear, and they are not symmetric. The state entering (33) is $B^k[t-1]$, produced by the *current* sweep a moment earlier, and it is imposed as a hard constraint. The value of the future enters only through the last term of (31), and both μ and B there carry the superscript $k-1$: they are numbers left over from the *previous* sweep.

2) What is carried between sweeps

After solving (31)–(34) the scheme reads the multiplier of the period's own dynamics,

$$\mu^k[t] = -\text{dual}((33)), \quad (35)$$

which is the marginal value of one more unit of stored energy at t . No separate dual update is performed: (35) is the whole of the feedback. Sweeps repeat until the trajectories settle,

$$\max\left(\|B^k - B^{k-1}\|_2, \|\mu^k - \mu^{k-1}\|_2\right) < \epsilon. \quad (36)$$

3) Correspondence, and the one term that is missing

$\mu[t]$ is the same object as the value gradient V_B^t of Section I-C: both are the marginal worth of stored energy at t , and (26) is the statement that a converged sweep would satisfy. The box multipliers of (34) correspond to $\lambda_{B \min}^t, \lambda_{B \max}^t$ and to $\nu_{\min}^t, \nu_{\max}^t$ exactly as before. The two schemes are therefore carrying the same quantities.

What has no counterpart is V_{BB}^t . The coupling term in (31) is *linear* in $B[t]$: it carries the slope of the cost-to-go evaluated at the previous sweep's state and nothing more. The true cost-to-go has curvature, so as $B[t]$ moves during a sweep the marginal value of stored energy moves with it, and a frozen slope cannot follow. Two consequences follow, both measured below: the optimum is not quite a fixed point of the sweep, and the iteration does not settle onto it.

4) What it does on this instance

On the $T = 6$ instance of Table II with all bounds imposed, the scheme is implemented exactly as (31)–(36) and started from $B^0[t] = B_0, \mu^0[t] = 0$.

The subproblem is right. Supplied with the true multipliers μ^* and states B^* taken from a centralized solve, a *single* sweep reproduces the centralized optimum to 1.7×10^{-8} in objective and 6×10^{-7} in P_B . The scheme's fixed point is the correct answer.

The iteration does not reach it. From the cold start above it settles into a period-two limit cycle: two objectives alternating indefinitely, the state residual of (36) pinned at 2.1×10^{-1} , still cycling after 300 sweeps, 2.1×10^{-3} short in objective and up to 121 kW out in dispatch. Damping the update, $x \leftarrow (1 - a)x^{k-1} + ax^k$, shrinks the cycle without closing it:

a	residual after 2000 sweeps	gap in objective
1.0	2.1×10^{-1}	2.1×10^{-3}
0.5	2.0×10^{-2}	2.9×10^{-4}
0.1	6.1×10^{-4}	3.2×10^{-5}

The reason is visible in the oracle test: started *at* the optimum, the sweep returns multipliers differing from μ^* by 9.2×10^{-4} . The optimum is very nearly, but not exactly, a fixed point of the linearised map, so damping only shrinks the neighbourhood the iterate circles in. For contrast, the second-order method of Section I-C reaches 2.2×10^{-10} on the identical instance in 16 iterations.

REFERENCES

- [1] A. R. Jha, S. Paul, and A. Dubey, "Analyzing the performance of linear and nonlinear multi-period optimal power flow models for active distribution networks," in *2025 IEEE North-East India International Energy Conversion Conference and Exhibition (NE-IECC)*. IEEE, 2025, pp. 04–06.

TABLE I
FILTERDDP ITERATIONS, ALL BOUNDS

k	objective	backward pass				forward pass	
		pr_∞	du_∞	cs_∞	$\log_{10} \mu$	α	bk
0	0.840000	$1.0e+00$	$9.2e-01$	$1.0e+00$	-1.40	0.50	1
1	1.225783	$5.0e-01$	$4.6e-01$	$3.8e-01$	-1.40	0.50	1
2	1.397002	$2.5e-01$	$2.3e-01$	$1.9e-01$	-2.10	0.50	1
3	1.464625	$1.2e-01$	$1.1e-01$	$9.6e-02$	-2.10	1.00	0
4	1.520935	$2.2e-16$	$7.8e-16$	$1.8e-02$	-2.80	0.50	1
5	1.507179	$2.6e-16$	$6.1e-16$	$1.1e-02$	-3.49	0.50	1
6	1.496237	$2.5e-16$	$1.2e-15$	$6.0e-03$	-3.49	0.50	1
7	1.488496	$2.2e-16$	$7.8e-16$	$3.5e-03$	-4.19	0.50	1
8	1.483187	$3.3e-16$	$1.1e-15$	$2.0e-03$	-4.19	0.50	1
9	1.479927	$3.3e-16$	$1.3e-15$	$1.1e-03$	-4.19	1.00	0
10	1.476349	$2.2e-16$	$7.5e-16$	$3.1e-04$	-5.03	1.00	0
11	1.475395	$2.2e-16$	$1.8e-15$	$7.0e-05$	-6.04	1.00	0
12	1.475195	$2.2e-16$	$3.5e-15$	$1.0e-05$	-6.04	1.00	0
13	1.475179	$4.4e-16$	$1.3e-14$	$1.4e-06$	-7.25	1.00	0
14	1.475173	$4.4e-16$	$1.9e-15$	$6.7e-08$	-8.70	1.00	0
15	1.475173	$2.2e-16$	$6.3e-15$	$2.0e-09$	-10.44	1.00	0
16	1.475173	$2.2e-16$	$1.1e-13$	$3.7e-11$	-10.44	—	—

The backward pass evaluates the residuals at the current iterate; the forward pass is the filter line search that follows it, with accepted step α and $\text{bk} = \log_2(1/\alpha)$ backtracks. FilterDDP prints these two halves one iteration out of step, because `iteration_status` is called before `forward_pass!`; the rows here are realigned so each holds one backward pass and the forward pass it actually fed. The last row has no forward pass — the solver converged instead of stepping. Regularisation never fired. Every backtrack in the table was a rollout rejection rather than a filter or sufficient-decrease rejection, and the solver's own `ls` counter records none of them.

TABLE II
INSTANCE DATA

Interval t	1	2	3	4	5	6
Demand p_L^t	1.657	1.888	1.998	1.834	1.623	1.657
Energy price c^t	0.140	0.197	0.175	0.105	0.083	0.140

Period length Δt	1	Initial energy B_0	2.0
Throughput C_B	0.05	Terminal weight w	5.0

<i>Bounds, constrained variants only</i>			
$P_B^{\min}, P_B^{\max}$		−0.45, 0.45	
$B^{\min}, B^{\max}$		1.20, 1.95	

All quantities are in per unit on the tADMM base, 1 p.u. = 1000 kW and 1 p.u.h = 1000 kWh; with $\Delta t = 1$, power and energy are numerically interchangeable, and the bounds are ± 450 kW and 1200–1950 kWh. Figs. 4 and 5 report battery quantities in kW and kWh rather than per unit. The series are the tADMM load and price profiles, $p_L^t = 2\lambda^t$ with $\lambda^t = 0.8 + 0.1(\sin(\theta^t - 0.8) + 1)$ and $c^t = 0.08 + 0.06(\sin \theta^t + 1)$, sampled at $\theta^t \in \text{range}(0, 2\pi, T)$. These *resample* with T , so a shorter horizon is not a prefix of this row; both endpoints of $[0, 2\pi]$ are included, which is why $c^1 = c^T$. The -0.8 phase offset on the load is what keeps the two series from being collinear; their Pearson correlation is 0.644. Bounds apply only to the constrained variants.

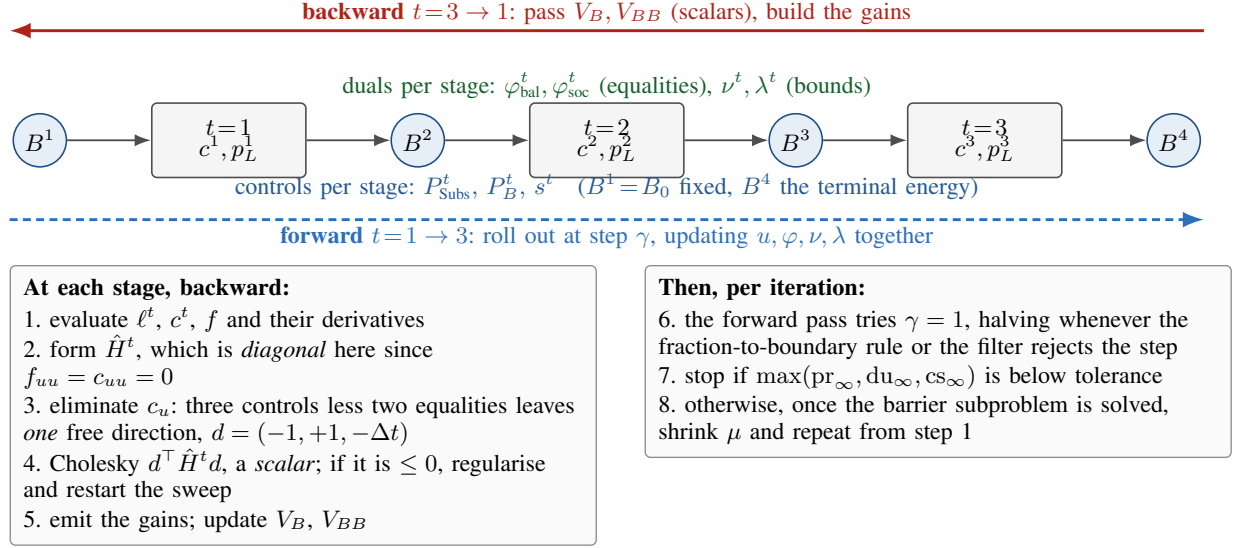


Fig. 1. How the copper-plate problem is actually solved, drawn at $T = 3$ so the tape fits; the structure is the same at any horizon, and the results reported here are $T = 6$. Read it as one worked example. Each stage owns its own price and demand, so nothing is interpolated and the state is the stored energy alone — which is why the cost-to-go derivatives V_B and V_{BB} carried backward along the top are single numbers rather than a vector and a matrix. The backward sweep builds a local model at each stage and leaves behind an affine rule; the forward sweep applies those rules left to right at one step size γ , moving the powers, the equality multipliers and the bound multipliers together. The stage solve is the interesting part: three controls against two equalities leave exactly one direction free, so the “dense” factorisation is a division by a single number, and the regularisation DDP is known for fires only if that number turns non-positive — which on this problem it never did.

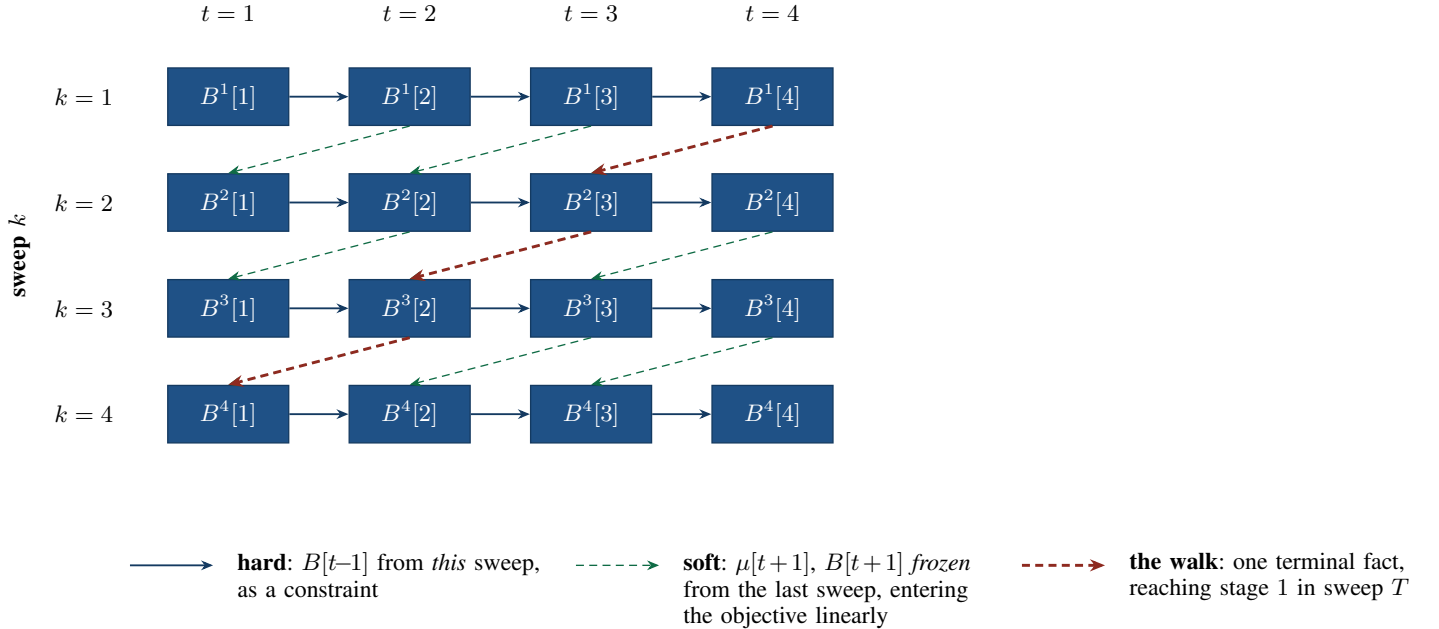


Fig. 2. What the first-order scheme passes between its subproblems, drawn for four stages over four sweeps. Each cell is one small quadratic program. Read one row at a time. Within a sweep the state moves left to right along the solid arrows and is always current — stage t is bound to the energy stage $t-1$ has just produced, as a hard constraint. The information travelling the other way cannot do that. Stage t learns what the future is worth only through $\mu[t+1]$ and $B[t+1]$ as they stood when the *previous* sweep finished, so that knowledge slides one column left per row and no faster. The heavy staircase follows a single fact on its way through: what the terminal stage discovers in sweep 1 does not reach stage 1 until sweep 4. A horizon of T periods therefore needs about T sweeps before the first stage has heard from the last, and each sweep costs T solver calls.

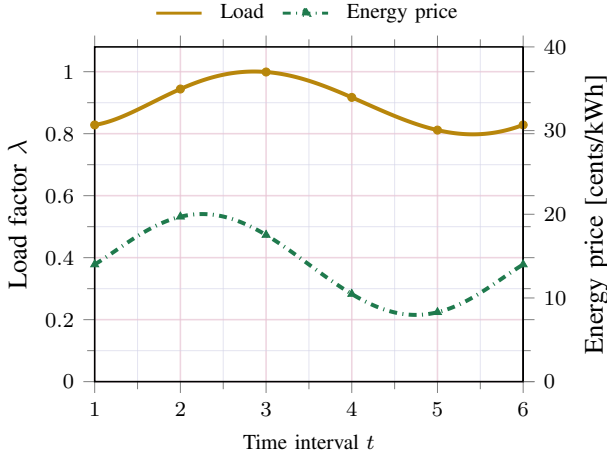


Fig. 3. Input profiles at $T = 6$: load factor (left axis), energy price (right). The markers are the data the solver uses — one price and one demand per stage, taken directly; the curves are the underlying tADMM profiles they are sampled from, drawn only to show the shape. The -0.8 rad phase offset between them is what keeps the two series from being collinear.

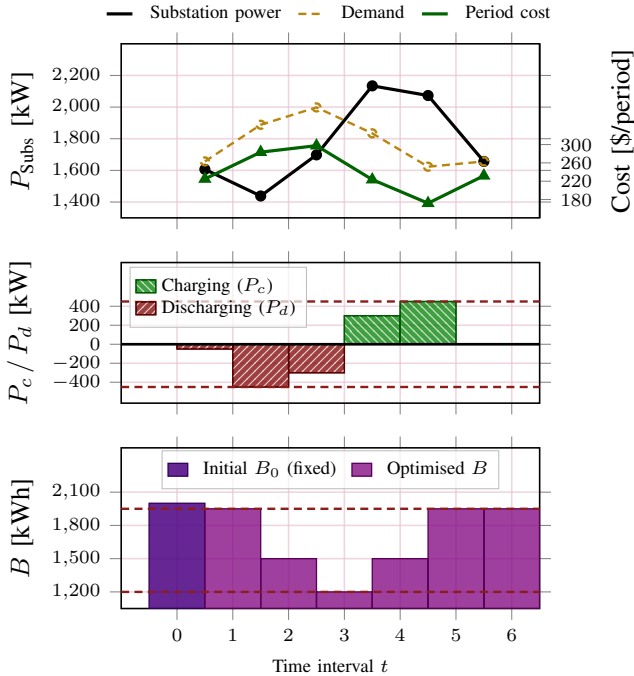


Fig. 4. The benchmark at $T = 6$ with all bounds imposed. Charging is drawn upward, discharging downward, so a bar's direction is the device's action rather than the sign of P_B^t . The battery discharges through $t = 1$ – 3 and recharges at $t = 4, 5$, riding its ± 450 kW rails and both energy limits (dashed). Those five bounds are the active set the reference solver reports.

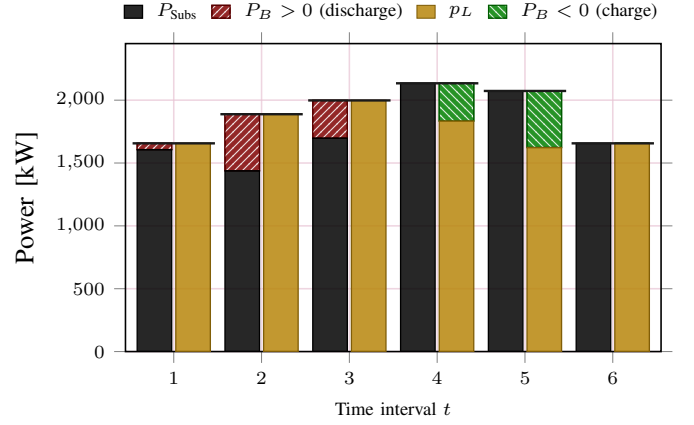


Fig. 5. Power-balance composition at $T = 6$. Per interval, supply (P_{Subs}^t plus discharge) on the left against demand (p_L^t plus charge) on the right; the equal heights are the power balance, marked by the rule. At $t = 2$ the battery supplies 23.8% of the node's 1888 kW; at $t = 5$ its charging instead raises import to 2073 kW to serve 1623 kW of load.

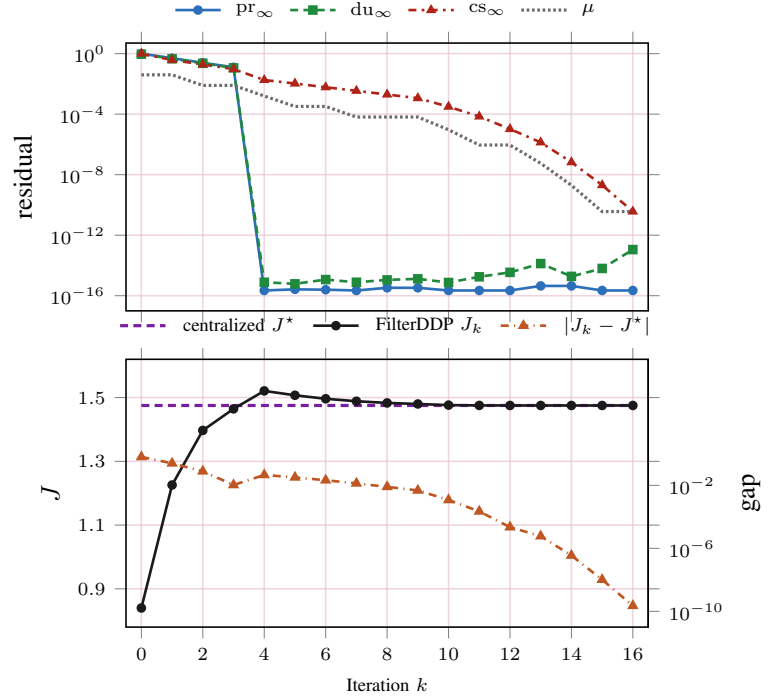


Fig. 6. FilterDDP convergence with all bounds imposed, 17 iterations. Top: the KKT residuals and the barrier parameter. Feasibility is reached almost immediately — pr_∞ and du_∞ hit machine precision by $k = 4$ — after which progress is entirely μ driving cs_∞ down alongside it, so the iteration count is set by the barrier schedule rather than by the optimisation. Bottom: the objective against the centralized JuMP/Ipopt baseline J^* , with the gap on the right axis. J_k is not monotone — it overshoots J^* near $k = 5$ and returns from above, because the line search decreases the barrier objective rather than J — and the gap closes to 2.3×10^{-10} .